RISC-V Processor with Spike Pattern Verification

Outline

- Spike Pattern Generation
  - Compile
  - Log Processing
- RV32IM Core Implementation
  - Supported Instructions
  - Schemetic

Spike Pattern Generation - Compile

Pattern 1: Matrix Multiplication

riscv32-unknown-elf-gcc -O3 -funroll-loops -march=rv32im -mabi=ilp32 test.c -o test.elf

```c
void matmul(int a[4][4], int b[4][4], int c[4][4]) {
    for (int i = 0; i < 4; i++) {
        for (int j = 0; j < 4; j++) {
            int sum = 0;
            for (int k = 0; k < 4; k++) {
                sum += a[i][k] * b[k][j];
            }
            c[i][j] = sum;
        }
    }
}

int main() {
    init_data();
    matmul(A, B, C);
    return 0;
}
```

Spike Pattern Generation - Compile

Pattern 2: Comprehensive Instruction Test.

riscv32-unknown-elf-gcc -O0 -march=rv32im -mabi=ilp32 all_inst.c -o all_inst.elf

```
// A1. Test I-Type Arithmetic
int a = 20;
int b = (a + 12) - 2; // addi
int c = (b ^ 0xAA) & 0x55; // xori, andi
int d = (c | 0x01); // ori

// A2. Test Shift Logic (Barrel Shifter & Fill Bit)
int neg = -16; // 0xFFFFFFF0
int sra_res = neg >> 2;  // srai: Expect 0xFFFFFFFC (-4)
unsigned int uneg = 0xFFFFFFF0;
unsigned int srl_res = uneg >> 2; // srli: Expect 0x3FFFFFFC
int sll_res = a << 3; // slli

// A3. Test Comparisons (Signed vs Unsigned)
int slt_s = (neg < a);   // slti (signed): Expect 1
int slt_u = ((unsigned int)a < uneg); // sltiu (unsigned): Expect 1
```

Spike Pattern Generation - Compile

Dump Instruction Memory

riscv32-unknown-elf-objdump -d test.elf > test.dis

```
000100e0 <main>:
   100e0:   ff010113            addi  sp,sp,-16
   100e4:   00112623            sw ra,12(sp)
   100e8:   110000ef            jal   101f8 <init_data>
   100ec:   da018513            addi  a0,gp,-608 # 13b78 <A>
   100f0:   08050613            addi  a2,a0,128
   100f4:   04050593            addi  a1,a0,64
   100f8:   214000ef            jal   1030c <matmul>
   100fc:   00c12083            lw ra,12(sp)
   10100:   00100793            li a5,1
   10104:   fef02823            sw a5,-16(zero) # fffffff0 <__BSS_END__+0xfffec0c8>
   10108:   00000513            li a0,0
   1010c:   01010113            addi  sp,sp,16
   10110:   00008067            ret

00010114 <register_fini>:
   10114:   00000793            li a5,0
   10118:   00078863            beqz  a5,10128 <register_fini+0x14>
   1011c:   00012537            lui   a0,0x12
   10120:   32450513            addi  a0,a0,804 # 12324 <__libc_fini_array>
   10124:   09c0106f            j  111c0 <atexit>
   10128:   00008067            ret
```

Spike Pattern Generation - Compile

Dump Log.

riscv-isa-sim/build/spike -l --log-commits --isa=RV32IMAC riscv-pk/build/pk all_inst.elf 2> golden_trace_all_inst.txt

```
core   0: 0x00010228 (0xf9010113) addi    sp, sp, -112
core   0: 0 0x00010228 (0xf9010113) x2  0x7ffffd30
core   0: 0x0001022c (0x06112623) sw      ra, 108(sp)
core   0: 0 0x0001022c (0x06112623) mem 0x7ffffd9c 0x0001015c
core   0: 0x00010230 (0x06812423) sw      s0, 104(sp)
core   0: 0 0x00010230 (0x06812423) mem 0x7ffffd98 0x00000000
core   0: 0x00010234 (0x07010413) addi    s0, sp, 112
core   0: 0 0x00010234 (0x07010413) x8  0x7ffffda0
core   0: 0x00010238 (0x01400793) li      a5, 20
core   0: 0 0x00010238 (0x01400793) x15 0x00000014
core   0: 0x0001023c (0xfef42423) sw      a5, -24(s0)
core   0: 0 0x0001023c (0xfef42423) mem 0x7ffffd88 0x00000014
core   0: 0x00010240 (0xfe842783) lw      a5, -24(s0)
core   0: 0 0x00010240 (0xfe842783) x15 0x00000014 mem 0x7ffffd88
core   0: 0x00010244 (0x00a78793) addi    a5, a5, 10
core   0: 0 0x00010244 (0x00a78793) x15 0x0000001e
core   0: 0x00010248 (0xfef42223) sw      a5, -28(s0)
core   0: 0 0x00010248 (0xfef42223) mem 0x7ffffd84 0x0000001e
core   0: 0x0001024c (0xfe442783) lw      a5, -28(s0)
core   0: 0 0x0001024c (0xfe442783) x15 0x0000001e mem 0x7ffffd84
core   0: 0x00010250 (0x0aa7c793) xori    a5, a5, 170
core   0: 0 0x00010250 (0x0aa7c793) x15 0x000000b4
```

Spike Pattern Generation - Log Processing

Start PC

```
00010228 <main>:
   10228:   f9010113            addi  sp,sp,-112
   1022c:   06112623            sw ra,108(sp)
   10230:   06812423            sw s0,104(sp)
   10234:   07010413            addi  s0,sp,112
   10238:   01400793            li a5,20
   1023c:   fef42423            sw a5,-24(s0)
```

```
   10394:   fec42703            lw a4,-20(s0)
   10398:   00f707b3            add   a5,a4,a5
   1039c:   fef42623            sw a5,-20(s0)
   103a0:   00000013            nop
   103a4:   06c12083            lw ra,108(sp)
   103a8:   06812403            lw s0,104(sp)
   103ac:   07010113            addi  sp,sp,112
   103b0:   00008067            ret
```

End PC

Spike Pattern Generation - Log Processing

Include only Log from start PC to end PC

```
core   0: 0x00010158 (0x0d0000ef) jal      pc + 0xd0
core   0: 0 0x00010158 (0x0d0000ef) x1  0x0001015c
core   0: 0x00010228 (0xf9010113) addi     sp, sp, -112
core   0: 0 0x00010228 (0xf9010113) x2  0x7ffffd30
core   0: 0x0001022c (0x06112623) sw       ra, 108(sp)
```

```
core   0: 0x000103ac (0x07010113) addi     sp, sp, 112
core   0: 0 0x000103ac (0x07010113) x2  0x7ffffda0
core   0: 0x000103b0 (0x00008067) ret
core   0: 0 0x000103b0 (0x00008067)
```

Spike Pattern Generation - Log Processing

Aquire register initial state.

```
bofan@DESKTOP-4UGCJAG:~/RISCV$ riscv-isa-sim/build/spike -d --isa=RV32IMAC riscv-pk/build/pk all_inst.elf
(spike) until pc 0 10228
(spike) reg 0
zero: 0x00000000  ra: 0x0001015c  sp: 0x7ffffda0  gp: 0x00013a78
  tp: 0x00000000  t0: 0x00010c84  t1: 0x0000000f  t2: 0x00000000
  s0: 0x00000000  s1: 0x00000000  a0: 0x00000001  a1: 0x7ffffda4
  a2: 0x7ffffdac  a3: 0x00000000  a4: 0x00000001  a5: 0x00000000
  a6: 0x00000000  a7: 0x00000000  s2: 0x00000000  s3: 0x00000000
  s4: 0x00000000  s5: 0x00000000  s6: 0x00000000  s7: 0x00000000
  s8: 0x00000000  s9: 0x00000000 s10: 0x00000000 s11: 0x00000000
  t3: 0x00000000  t4: 0x00000000  t5: 0x00000000  t6: 0x00000000
```

Verification Flow

- Initialize register
  - lui/addi
  - or direct assignment
- Reset PC to “start PC” and start simulation
- Compare all the expected register write/SW/LW with the actual register write/SW/LW
- Stop simulation when “end PC” is reached.

RV32IM Implementation - Supported Instructions

Category	Supported Instructions
Arithmetic (I-Type)	ADDI, SLTI, SLTIU, XORI, ORI, ANDI
Arithmetic (R-Type)	ADD, SUB, SLL, SLT, SLTU, XOR, SRL, SRA, OR, AND
Logical Shifts	SLLI, SRLI, SRAI
Memory Access	LW, SW
Branches	BEQ, BNE, BLT, BGE, BLTU, BGEU
Jumps	JAL, JALR
Upper Imm	LUI, AUIPC
M-Extension	MUL

RV32IM Implementation - Schemetic